

Git & Docker Implementation Report

CarbonSense

AI-Based Personal Carbon Footprint Tracking and Sustainability Advisor

Submitted in partial fulfilment for the course of

CSA1012 — Software Engineering

Git Repository Management & Docker Containerization

Submitted by

Deepak Kumar A

192524422

GitHub Repository: <https://github.com/dhepakk08/CarbonSense>

Under the supervision of

Dr. K. Anitadavamani

1. Project Structure Design

Before initializing version control, the CarbonSense project was organized into a clear, modular folder structure that separates presentation, business logic, and data concerns. This separation keeps the Git history readable (changes to the frontend, backend, and configuration stay logically distinct) and maps directly onto the Docker services introduced later in this report.

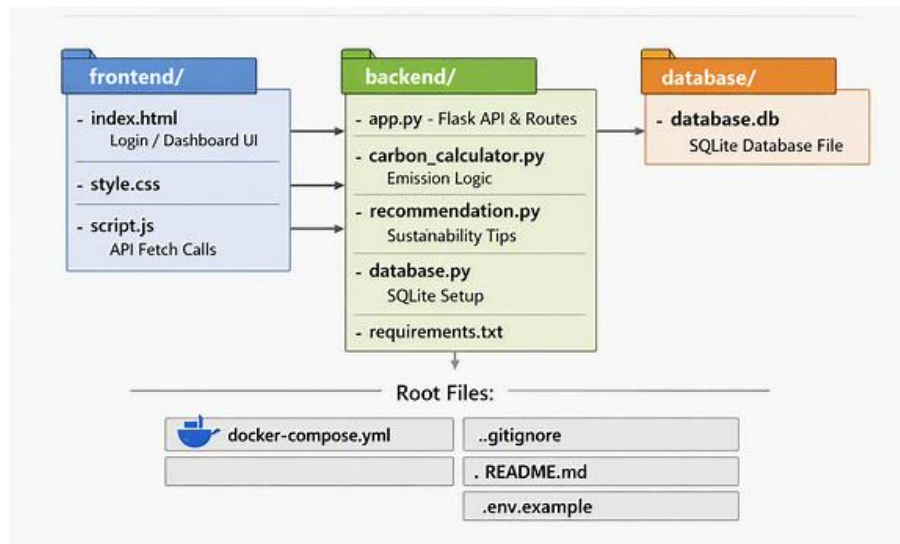


Figure 1: CarbonSense project folder structure.

Folder / File	Purpose
frontend/	Presentation layer — index.html (page structure), style.css (styling), script.js (view logic and API calls to the backend).
backend/	Application layer — app.py (Flask routes), carbon_calculator.py (emission logic), recommendation.py (sustainability tips), database.py (SQLite setup), requirements.txt.
database/	Holds database.db, the SQLite file that stores users, activities, and goals.
docker-compose.yml	Root-level orchestration file that builds and runs the frontend and backend containers together.
.gitignore	Excludes local/generated files (e.g. __pycache__, .env) from version control.
README.md	Project overview and setup instructions for anyone cloning the repository.

This structure was decided before running git init so that the very first commit already reflected an organized, review-ready layout rather than an unstructured dump of files.

2. Git Repository Initialization

With the folder structure in place, the local project was converted into a Git repository and connected to a remote GitHub repository. The steps below were carried out in PowerShell from the project root, and each is backed by a screenshot of the actual command output.

2.1 Create the Project Directory

The project folders (Back-end/ and Front-end/) were first created on disk inside the working directory used for this course.

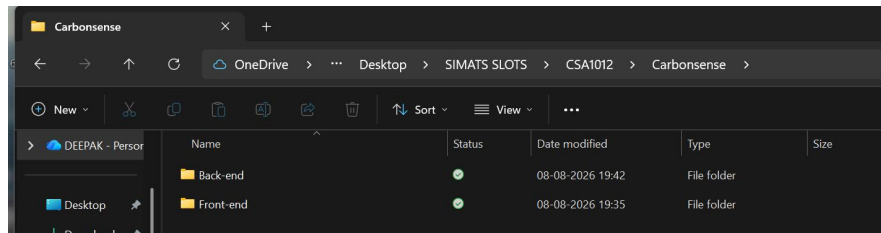


Figure 2: Project directory created in File Explorer.

2.2 Initialize the Local Repository

`git init`

This command creates a hidden `.git` folder that turns the project directory into a Git-tracked repository, enabling version history from this point forward.

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git init
Initialized empty Git repository in C:/Users/Lenovo/OneDrive/Desktop/SIMATS SLOTS
/CSA1012/Carbonsense/.git/
```

Figure 3: Output of `git init`.

2.3 Stage All Project Files

`git add .`

Stages every file in the working directory for the first commit. The line-ending warnings shown (LF will be replaced by CRLF) are expected on Windows and do not affect file content — Git is simply normalizing line endings for cross-platform consistency.

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git add .
warning: in the working copy of 'Back-end/app.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'Back-end/carbon_calculator.py', LF will be repla
ced by CRLF the next time Git touches it
warning: in the working copy of 'Back-end/database.py', LF will be replaced by CR
LF the next time Git touches it
warning: in the working copy of 'Back-end/recommendation.py', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'Back-end/requirements.txt', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'Front-end/index.html', LF will be replaced by CR
LF the next time Git touches it
warning: in the working copy of 'Front-end/script.js', LF will be replaced by CRL
F the next time Git touches it
warning: in the working copy of 'Front-end/style.css', LF will be replaced by CRL
F the next time Git touches it
```

Figure 4: Output of `git add`.

2.4 Commit the Initial Snapshot

```
git commit -m "Initial project setup"
```

Records the staged files as the first commit in the project's history, with a clear, descriptive message — the first of several conventions followed throughout this project to keep the commit history readable.

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git commit
-m "Initial project setup"
[master (root-commit) 4831c39] Initial project setup
 8 files changed, 1696 insertions(+)
 create mode 100644 Back-end/app.py
 create mode 100644 Back-end/carbon_calculator.py
 create mode 100644 Back-end/database.py
 create mode 100644 Back-end/recommendation.py
 create mode 100644 Back-end/requirements.txt
 create mode 100644 Front-end/index.html
 create mode 100644 Front-end/script.js
 create mode 100644 Front-end/style.css
```

Figure 5: Output of `git commit`.

2.5 Connect to the Remote Repository

A repository named CarbonSense was created on GitHub, and the local repository was linked to it as the remote named origin.

```
git remote add origin https://github.com/dhepakk08/CarbonSense.git
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git remote
add origin https://github.com/dhepakk08/CarbonSense.git
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git remote
-v
origin https://github.com/dhepakk08/CarbonSense.git (fetch)
origin https://github.com/dhepakk08/CarbonSense.git (push)
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense>
```

Figure 6: Remote origin added and verified.

2.6 Rename the Default Branch

```
git branch -M main
```

Ensures the primary branch is named main, matching GitHub's current default and the branching strategy described in Section 3.

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git branch
-M main
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git branch
* main
```

Figure 7: Branch renamed to main.

2.7 Push to GitHub

git push -u origin main

Uploads the initial commit to GitHub and sets main to track origin/main, so future git push / git pull commands need no further arguments.

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git push -u
origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (12/12), 17.30 KiB | 1.57 MiB/s, done.
Total 12 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/dhepakk08/CarbonSense.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 8: Initial push to the remote repository.

2.8 Verify the Repository

git log --oneline

A final check confirms the commit is present locally and that HEAD, main, and origin/main all point to the same commit — the repository was ready for active development.

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git log --o
neline
4831c39 (HEAD -> main, origin/main) Initial project setup
```

Figure 9: Verified commit history after the initial push.

The complete initialization sequence is summarized in Figure 10 below.

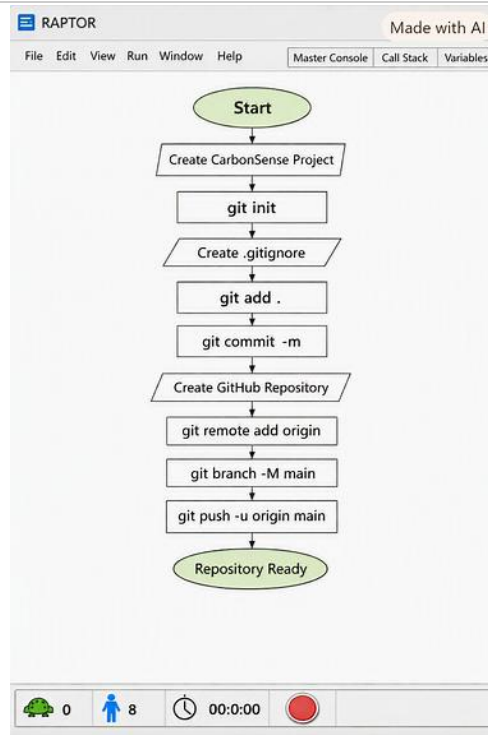


Figure 10: Repository initialization workflow.

3. Git Branching Strategy

CarbonSense follows a lightweight Main → Development → Feature branching model. main always holds a stable, deployable snapshot of the project; development integrates finished work before it reaches main; short-lived feature/* branches isolate individual pieces of work (e.g. the frontend UI, the Flask API) so they can be built and tested without disturbing the rest of the codebase.

Branch	Purpose	Example command
main	Stable, always-deployable code	git checkout main
development	Integration branch for completed features	git checkout -b development
feature/*	One branch per unit of work (e.g. feature/frontend-ui)	git checkout -b feature/frontend-ui

3.1 Branching Workflow

The sequence below — create the branch, commit work on it, push it, then open a pull request into development — was used consistently for every feature added to the project.

```

git checkout -b development
git push -u origin development
git checkout -b feature/frontend-ui
...work, add, commit...

```



```
git push -u origin feature/frontend-ui
# Pull request: feature/frontend-ui → development
# Pull request: development → main (after testing)
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git branch
* main
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git log --o
neline
4831c39 (HEAD -> main, origin/main) Initial project setup
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git checkou
t -b development
Switched to a new branch 'development'
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git branch
* development
  main
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> git push -u
origin development
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'development' on GitHub by visiting:
remote:   https://github.com/dhepakk08/CarbonSense/pull/new/development
remote:
To https://github.com/dhepakk08/CarbonSense.git
 * [new branch]      development -> development
branch 'development' set up to track 'origin/development'.
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense> █
```

Figure 11: Branch creation and push, verified in PowerShell.

3.2 Why This Strategy Fits CarbonSense

Principle	Benefit
Feature isolation	Frontend and backend work proceed independently without one breaking the other.
Stable main branch	main is never touched directly, so it always reflects a working version of the app.
Controlled integration	Pull requests into development give a review point before code reaches main.
Easier collaboration	Clear branch ownership avoids conflicting edits on the same files.
Traceable history	Each feature's commits stay grouped, making git log and code review easier to follow.
Safe experimentation	A feature branch can be discarded without any impact on main or development.

4. Docker Environment Design

CarbonSense is containerized as two cooperating services rather than one monolithic image, mirroring the frontend/backend split already established in the project structure. A frontend container (nginx) serves the static site and reverse-proxies API calls; a backend

container (Flask + gunicorn) implements the application logic; both are joined by a Docker-managed bridge network, and the SQLite database is kept outside both containers via a bind-mounted volume so data survives rebuilds.

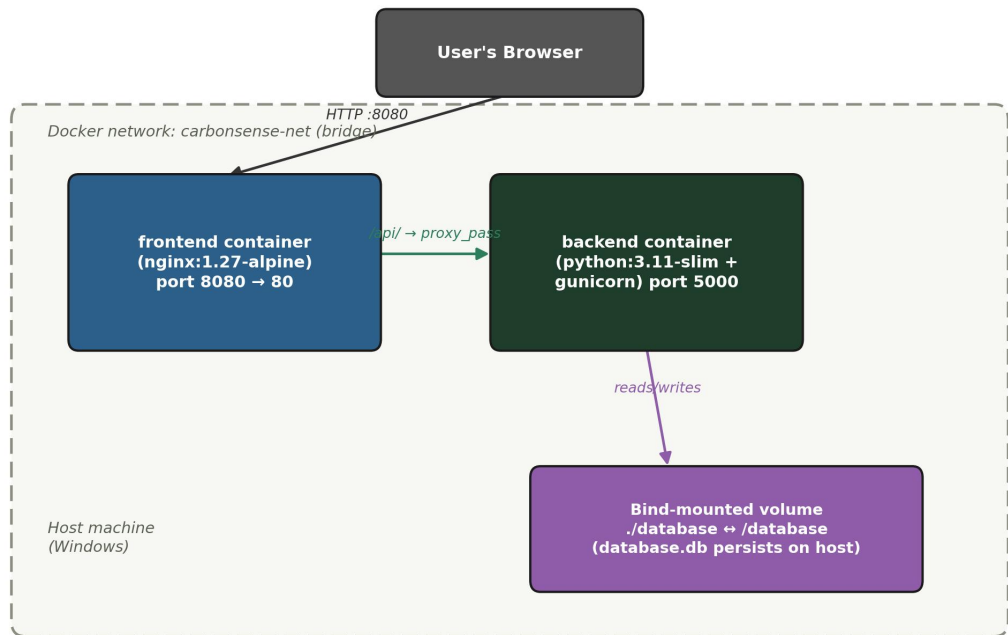


Figure 12: CarbonSense Docker environment — two services, one network, one persisted volume.

Design decision	Reason
Two containers, not one	Keeps the static frontend and the Python backend independently buildable, scalable, and replaceable.
nginx reverse proxy for /api/	The browser only ever talks to one origin (port 8080), which avoids CORS handling entirely in production.
Bind-mounted database volume	database.db lives on the host, not inside a container's writable layer, so data is not lost when a container is rebuilt or removed.
HEALTHCHECK on both images	Lets Docker (and docker compose ps) report real service health, not just “process is running.”

5. Dockerfile Development

Each service has its own Dockerfile, kept inside that service's folder so image build instructions travel with the code they build.

5.1 Backend Dockerfile

Based on python:3.11-slim for a small image footprint. Dependencies are installed before the application code is copied in, so Docker's layer cache is reused on every rebuild

unless requirements.txt itself changes — this is the single biggest factor in keeping repeated builds fast during development.

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    curl sqlite3 && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD curl -f http://localhost:5000/api/health || exit 1

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

5.2 Frontend Dockerfile

Based on nginx:1.27-alpine — a minimal image that serves the static site directly with no build step, since the frontend is plain HTML/CSS/JS.

```
FROM nginx:1.27-alpine
RUN rm -rf /usr/share/nginx/html/*
COPY index.html style.css script.js /usr/share/nginx/html/
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD wget --spider -q http://localhost/ || exit 1

CMD ["nginx", "-g", "daemon off;"]
```

6. Docker Compose Design

docker-compose.yml at the project root defines and links both services so the entire stack starts with a single command instead of manually building and running each container and wiring a network by hand.

```
services:
  backend:
    build: { context: ./backend, dockerfile: Dockerfile }
    container_name: carbonsense-backend
    ports: ["5000:5000"]
    volumes: ["/database:/database"]
    healthcheck:
```

```

test: ["CMD", "curl", "-f", "http://localhost:5000/api/health"]
interval: 30s

frontend:
  build: { context: ./frontend, dockerfile: Dockerfile }
  container_name: carbonsense-frontend
  ports: ["8080:80"]
  depends_on:
    backend: { condition: service_healthy }

```

The `depends_on: condition: service_healthy` clause is deliberate: without it, Compose only waits for the backend container to start, not for Flask/gunicorn inside it to actually be ready — this line ensures the frontend (and its nginx proxy) never comes up before the API can answer requests.

7. Container Deployment and Testing

With both Dockerfiles and `docker-compose.yml` in place, the full stack was built and deployed from the project root, then verified in three stages: (1) confirm both images built and both containers are running and healthy, (2) confirm the application is actually reachable and functioning from a browser, (3) shut the stack down cleanly.

7.1 Build and Start the Stack

`docker compose up --build -d` builds both images from their Dockerfiles and starts both containers in the background.

```

PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense\Front-end> docker compose up --build
-d
[+] up 5/5
✓ Image carbonsense-backend      Built      15.1s
✓ Image carbonsense-frontend     Built      15.1s
✓ Network carbonsense_default    Created    0.1s
✓ Container carbonsense-backend  Healthy   6.7s
✓ Container carbonsense-frontend Started    6.9s

```

7.2 Verify the Images

`docker images` lists every image built on the machine, confirming `carbonsense-backend` and `carbonsense-frontend` were created successfully.

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense\Front-end> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
carbonsense-backend:latest	67365e319947	216MB	53MB	U
carbonsense-frontend:latest	a3e59da40071	73.7MB	21MB	U
example:latest	06adcd704bda	1.6GB	413MB	
hello-world:latest	c3cbe1cc1aa5	25.9kB	9.49kB	U
mywebsite:latest	67c2f799a14c	92.8MB	26.1MB	U
python:3.13	6968f3db2567	1.62GB	430MB	U
ubuntu:latest	3131b4cc82a7	161MB	45.3MB	U

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense\Front-end>
```

7.3 Verify the Running Containers

docker ps lists active containers. The STATUS column should show Up, and once the HEALTHCHECK defined in each Dockerfile passes, (healthy).

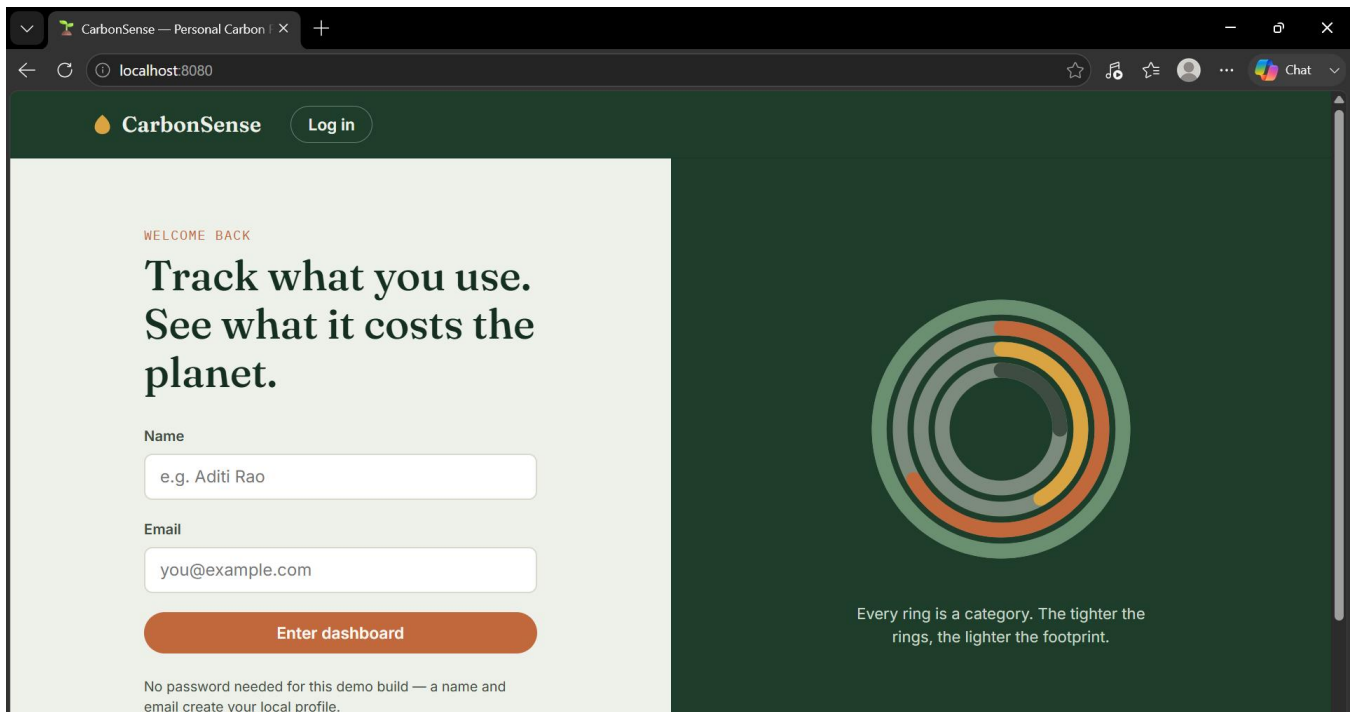
```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense\Front-end> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
12eb185eadc6	carbonsense-frontend	"/docker-entrypoint..."	22 seconds ago	Up 16 seconds (health: starting)
37829c5028d0	carbonsense-backend	"gunicorn --bind 0.0..."	22 seconds ago	Up 22 seconds (healthy)

```
PS C:\Users\Lenovo\OneDrive\Desktop\SIMATS SLOTS\CSA1012\Carbonsense\Front-end>
```

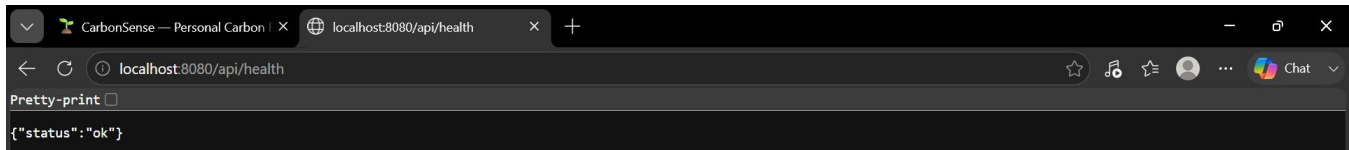
7.4 Functional Test — Application Loads

With both containers healthy, http://localhost:8080 was opened in a browser to confirm nginx is correctly serving the CarbonSense frontend.



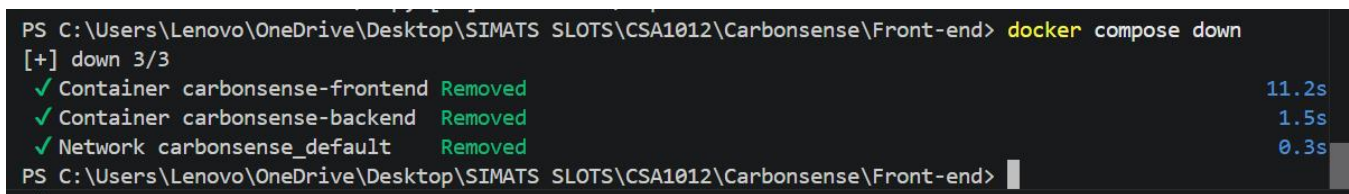
7.5 Functional Test — API Reachable Through the Proxy

`http://localhost:8080/api/health` was requested next. A successful `{"status": "ok"}` response proves nginx's `/api/` location block is correctly forwarding requests to the backend container over the internal Docker network — not just serving static files.



7.6 Shut Down the Stack

`docker compose down` stops and removes both containers (and the network Compose created) without touching the bind-mounted database, which remains on the host.



8. Benefits of Git and Docker

Used together, Git and Docker solve two different but complementary problems for CarbonSense: Git tracks how the source code changes over time; Docker guarantees the environment that code runs in is identical everywhere it is deployed.

Aspect	Git	Docker
Consistency	Every change is recorded; the project can be restored to any prior commit.	The app runs identically on any machine with Docker installed, regardless of local Python/nginx versions.
Collaboration	Branches let multiple people work in parallel without overwriting each other's changes.	Teammates run <code>docker compose up</code> and get a working stack with no manual setup steps.
Recovery	<code>git log</code> / <code>git checkout</code> make it possible to inspect or revert to any earlier state.	A broken container can be discarded and rebuilt from the Dockerfile in seconds.
Deployment	A tagged commit marks exactly what code is in a given release.	An image built from that commit can be shipped and run anywhere unchanged.

9. Challenges and Improvements

Challenge encountered	Resolution / future improvement
CRLF/LF line-ending warnings on git add	Expected behavior on Windows; a .gitattributes file could normalize line endings explicitly in a future iteration.
Frontend needing to reach the backend without CORS issues	Solved with an nginx reverse-proxy rule for /api/, so the browser only ever sees one origin.
SQLite data being lost on container rebuild	Solved with a bind-mounted volume (./database:/database) so the database file lives outside the container's writable layer.
Backend not “ready” the instant its container starts	Solved with a Dockerfile HEALTHCHECK plus depends_on: condition: service_healthy in Compose.
Image size and build time	Future improvement: multi-stage builds and a smaller base image to further reduce the backend image size.
No automated testing before deployment	Future improvement: a CI/CD pipeline (e.g. GitHub Actions) to run tests and build images automatically on every push.

Overall, combining Git's change-tracking with Docker's environment consistency gave CarbonSense a workflow where code, configuration, and runtime environment are all version-controlled and reproducible — a foundation the project can build on as it grows beyond a single-developer, single-machine setup.